

Research Note: Why Low-Level Design Practice is Broken & How to Fix It

Candidate: Priyanshu

Assignment: LLD Practice Platform — Engineering Assignment

Date: September 2026

1. The Core Problem: Why Practicing LLD is Frustrating

When practicing Data Structures & Algorithms (DSA), the feedback loop is crystal clear: your code either passes the hidden test cases or it fails.

Low-Level Design (LLD) is completely different. When an engineer sits down to design a **Parking Lot**, an **Elevator System**, or a **Splitwise app**, they face three big questions:

- 1. Did I split responsibilities cleanly, or did I accidentally build a massive God-class that handles everything?*
- 2. Is my design actually extensible, or will adding one new requirement break all my classes?*
- 3. If my solution looks completely different from the solution on GitHub, is mine actually wrong, or is it just a valid alternative trade-off?*

Because there are no automated test fixtures that can grade "good Object-Oriented Design," learners are left guessing. They usually end up memorizing static solutions from YouTube or GitHub without understanding *why* specific design choices were made.

2. What I Found When Researching Existing Tools

I spent time analyzing how developers currently prepare for LLD interviews and looked at the four most common options:

1. Static GitHub Repositories (e.g., "awesome-low-level-design")

- **What people do:** Read someone else's Java/C++ repository and try to recreate it from memory.
- **The Problem:** It reinforces the idea that there is only **one golden solution**. If you used the Strategy Pattern instead of the State Pattern, you have no way of knowing whether your choice was reasonable. It teaches memorization rather than design thinking.

2. LeetCode "Design" Problems (e.g., Design Underground System, LRU Cache)

- **What people do:** Write code to satisfy specific function inputs and outputs.
- **The Problem:** LeetCode only checks input/output correctness. You can pass the test suite with a 400-line monolithic class filled with nested HashMaps and raw integers. It rewards quick algorithmic hacks and completely ignores SOLID principles, encapsulation, and clean abstractions.

3. Asking General-Purpose AI (ChatGPT / Claude / Copilot)

- **What people do:** Paste their code and ask *"Is this a good LLD design?"*
- **The Problem:** Unstructured AI feedback is usually overly polite, giving generic compliments and arbitrary nitpicks (like variable naming). It rarely quotes specific lines of code as evidence, changes its opinion if you ask twice, and doesn't grade against a consistent standard.

4. Paid Mock Interviews (Pramp / Interviewing.io)

- **What people do:** 45-minute live sessions with a senior engineer.
 - **The Problem:** Highly valuable, but too expensive and high-friction for daily, iterative practice.
-

3. The Three Major Gaps

From this research, three missing pieces became clear:

1. **Feedback Must Cite Real Evidence:** General comments like "*your design is tightly coupled*" aren't helpful. Feedback needs to point directly to the learner's code:

"Class `ParkingFloor` directly instantiates `HourlyPricingStrategy` on line 12 instead of taking an `IPricingStrategy` interface in its constructor."

1. **There is No Single "Right" Answer:** Two completely different designs can both be good if their trade-offs make sense for the problem's constraints. Evaluation must grade how well the solution satisfies the requirements, not whether it matches a template.

1. **Practice Requires an Iteration Loop, Not a One-Time Test:** The entire point of practice is:

Attempt #1 → Inspect Weaknesses → Fix Them in Attempt #2 → Measure Improvement

Current tools treat an attempt as a dead end. Learners need to see a side-by-side comparison of what changed between attempts.

4. Product Direction & Approach

To build a focused, practical MVP, I designed the product around three core ideas:

A. A Structured 5-Part Design Canvas

Instead of giving the user a blank code editor (which encourages jumping straight into writing methods without thinking), the canvas asks for the five things an interviewer actually looks for:

1. **Assumptions & Scope:** What is in scope and what is left out?
2. **Class Structure / Model:** What are the domain entities, attributes, and relationships?
3. **Design Patterns & Rationale:** Why was a pattern chosen over simple logic?
4. **Implementation Code:** Clean interfaces, access modifiers, and core methods.
5. **Trade-offs & Concurrency:** How are thread-safety and race conditions handled?

B. A Fixed 6-Dimension Rubric

Every submission is evaluated against 6 concrete design dimensions:

- Requirement Understanding (15%)
- Class Responsibilities & Cohesion / SRP (20%)
- Coupling, Interfaces & Encapsulation (20%)
- Appropriate Use of Design Patterns (15%)
- Extensibility & Trade-offs (15%)
- Edge Cases, Concurrency & Testability (15%)

For each dimension, the feedback must provide:

C. Attempt History & Delta Comparison

Every attempt is saved. When a learner submits a second attempt after fixing their code, they can open a **Comparison View** that shows exact score deltas per dimension, confirming that they actually resolved the previous flaws.

5. Summary

By structuring the submission, evaluating against an evidence-based rubric, and letting learners measure improvement across attempts, this platform turns Low-Level Design from subjective guesswork into a practical, repeatable learning loop.